

I will be combining my methodology, results, and discussion of results all together.

Question 10.1

Using the same crime data set `uscrime.txt` as in Questions 8.2 and 9.1, find the best model you can using

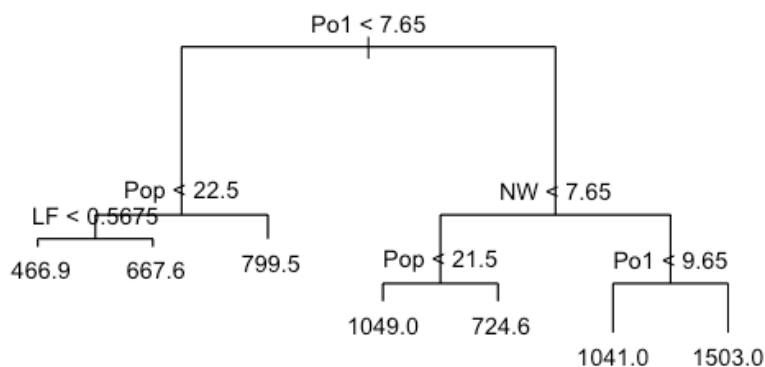
(a) a regression tree model, and

I. First, let's use the `tree` function on the whole dataset and look at the summary of the tree.

```
Regression tree:
tree(formula = Crime ~ ., data = crime)
Variables actually used in tree construction:
[1] "Po1" "Pop" "LF" "NW"
Number of terminal nodes: 7
Residual mean deviance: 47390 = 1896000 / 40
Distribution of residuals:
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-573.900 -98.300  -1.545   0.000 110.600  490.100
```

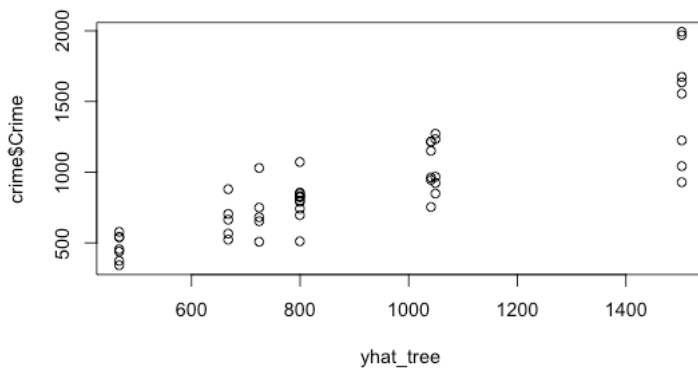
The summary of the tree tells us the number of terminal nodes, the residual mean deviance, and the distribution of residuals. The lower the residual mean deviance, the better the fit. If there are less terminal nodes, then the model is simpler, leading to risk of underfitting while more terminal nodes mean the model is more complex, leading to risk of overfitting; Therefore, we can prune the tree and choose the smallest tree that achieves minimum deviance.

II. Next, I plotted the tree model. After that, I used `frame` and `where` functions on `uscrime_tree` to display the observation index on the first row and the leaf IDs on the second row (which terminal node each observation ended up in).



As you can see in the tree diagram above, there are seven different terminal nodes on the bottom. In the numerical output, it shows the observation index and leaf IDs. For example, the first observation falls into leaf 6, the second observation falls into leaf 13, and so on.

- III. Then, I graphed a plot of the prediction of the tree model on the x-axis and the response variable on the y-axis. I calculated the R-squared value of the model by using the formula.



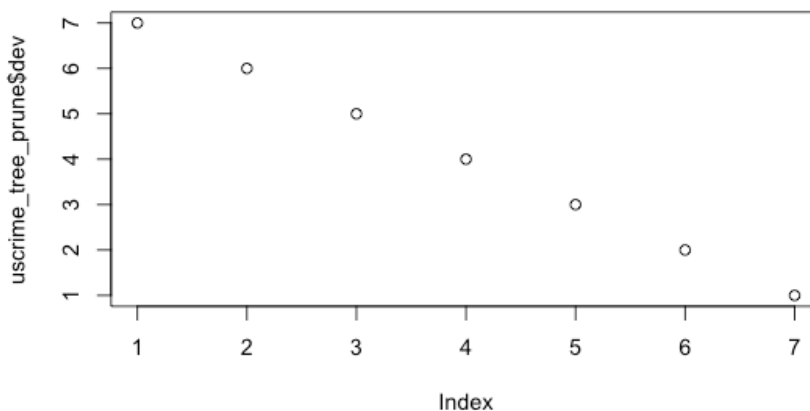
```
[1] 0.7244962
```

The R-squared value is 72.44962%. This is a decent R-squared value. 72.44962% of the variance is explained by the tree model.

- IV. I used `prune.tree` to get the number of terminal nodes and deviance of each.

```
[1] 7 6 5 4 3 2 1
[1] 1895722 2013257 2276670 2632631 3364043 4383406 6880928
```

Based on the deviance results above, at leaf 7, the deviance is the lowest. This means that it is a good fit and the tree's predictions are closer to the actual values. The tree's structure captures the patterns and relationships within the data. Therefore, I stuck with the original tree model (7 terminal nodes).



Our final regression tree model is `uscrime_tree_prune`, which uses seven terminal nodes, because at leaf 7, the deviance is lowest, and the R-Squared value is at a good value.

(b) a random forest model.

In R, you can use the `tree` package or the `rpart` package, and the `randomForest` package. For each model, describe one or two qualitative takeaways you get from analyzing the results (i.e., don't just stop when you have a good model, but interpret it too).

- I. First, I chose a number of predictors to test. I chose to use four predictors. Then, I used `randomForest()` where `mtry` was equal to the number of predictors, importance equal to `true`, and the number of trees equal to 500. Here is the output:

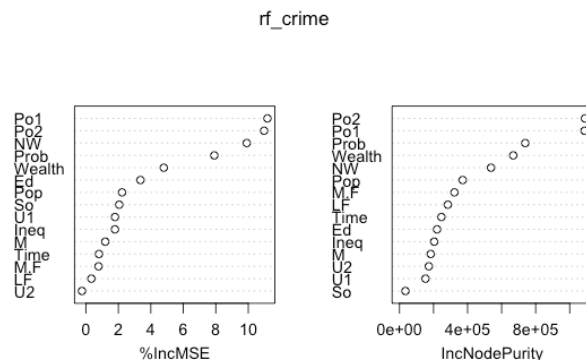
```
Call:
randomForest(formula = Crime ~ ., data = crime, mtry = num_pred, importance = TRUE, ntree = 500)

Type of random forest: regression
Number of trees: 500
No. of variables tried at each split: 4

Mean of squared residuals: 84238.14
% Var explained: 42.46
```

- II. Next, I used `importance()` on my `randomForest()` model to display the `%IncMSE` and `IncNodePurity` of each variable. I also used `varImpPlot()` to create two separate plots the output.

	%IncMSE	IncNodePurity
M	1.1952321	183407.18
So	2.0524861	34868.38
Ed	3.3623124	220557.25
Po1	11.1795386	1087197.60
Po2	10.9746617	1089675.70
LF	0.3400326	283829.97
M.F	0.7805245	322706.46
Pop	2.2273081	371194.62
NW	9.9052831	537548.06
U1	1.7925327	152625.75
U2	-0.2390979	172185.85
Wealth	4.7970483	668237.33
Ineq	1.7901634	203860.03
Prob	7.9078402	738705.58
Time	0.8062923	246166.16



Based on this output, the important variables are `Po1`, `Po2`, `NW`, and `Prob`. The reason for this is that the `%IncMSE` is the highest for those four variables. `%IncMSE` represents the percent increase in mean squared error, and it is important for prediction accuracy. They are also in the top 5 for `IncNodePurity`, which is important for explaining variance in training data.

- III. I calculated the R-squared of my random forest model.

```
[1] 0.4289338
```

The R-squared of `rf_crime` is 42.46135%. This means that 42.46135% of the variance is explained by the random forest model.

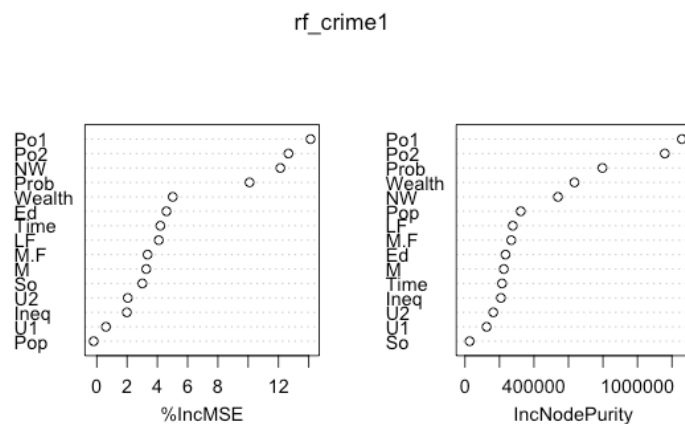
IV. I created a new model, but I chose to use 5 predictors. This is because using the number of predictors divided by 3 is a good value to test. Here is the output:

```
Call:
  randomForest(formula = Crime ~ ., data = crime, mtry = num_pred, importance =
TRUE, ntree = 500)
      Type of random forest: regression
      Number of trees: 500
No. of variables tried at each split: 5

      Mean of squared residuals: 84022.61
      % Var explained: 42.61
```

V. I used importance() and varImpPlot() on the new random forest model with five predictors.

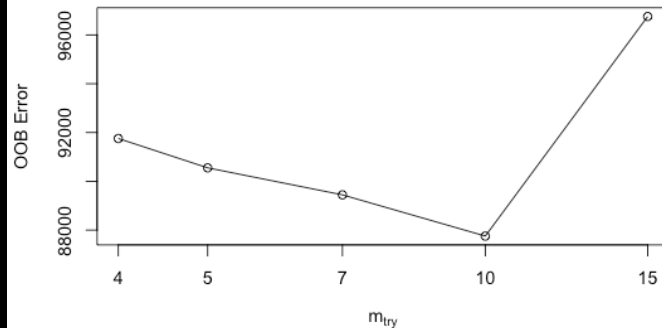
	%IncMSE	IncNodePurity
M	3.2674616	224980.91
So	3.0087380	26787.75
Ed	4.6037485	234969.59
Po1	14.1171682	1257578.10
Po2	12.6604864	1158474.59
LF	4.0944629	277019.40
M.F	3.3533690	268142.61
Pop	-0.2050245	323342.15
NW	12.1238763	538998.85
U1	0.6060503	125281.25
U2	2.0459769	164038.11
Wealth	5.0168386	634677.34
Ineq	1.9869769	208394.42
Prob	10.0889133	796843.42
Time	4.1984764	214713.85



I got the same important predictors as the first model. However, the values of the %IncMSE and IncNodePurity are higher than the first model's. The R-squared value was a bit higher, by about .2 percent. Therefore, using five predictors is better than four.

VI. I decided to use tuneRF() to help me decide how many predictors to use.

```
mtry = 5 OOB error = 90551.87
Searching left ...
mtry = 4 OOB error = 91754.29
-0.01327885 0.01
Searching right ...
mtry = 7 OOB error = 89444.21
0.01223237 0.01
mtry = 10 OOB error = 87753.47
0.0189027 0.01
mtry = 15 OOB error = 96755.19
-0.1025796 0.01
  mtry OOBError
4      4 91754.29
5      5 90551.87
7      7 89444.21
10     10 87753.47
15     15 96755.19
```



As you can see in this output, the OOB Error is lowest when $mtry = 10$. OOB Error is the mean squared prediction error calculated on "out-of-bag" samples.

VII. Therefore, I created a random forest model with ten predictors. Here is the output:

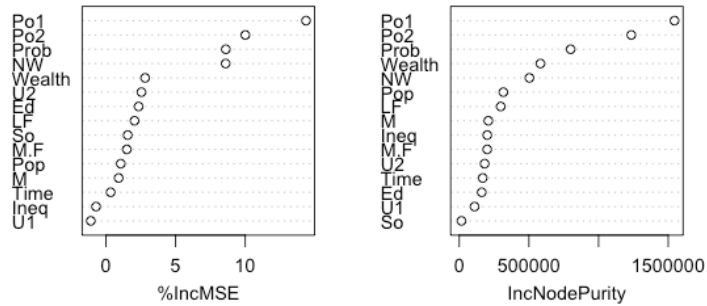
```
Call:
  randomForest(formula = Crime ~ ., data = crime, mtry = num_pred, importance = TRUE, ntree = 500)
      Type of random forest: regression
      Number of trees: 500
No. of variables tried at each split: 10

      Mean of squared residuals: 93904.75
      % Var explained: 35.86
```

VIII. I used `importance()` and `varImpPlot()` on the new model.

	%IncMSE	IncNodePurity
M	0.9083481	208572.0
So	1.5572726	15750.0
Ed	2.3382281	161148.8
Po1	14.3640887	1544974.4
Po2	10.0059416	1234953.8
LF	2.0483668	297469.2
M.F	1.4957394	200413.4
Pop	1.0565573	316398.8
NW	8.5907768	503498.8
U1	-1.0946171	109739.5
U2	2.5446818	182073.5
Wealth	2.8064272	582634.1
Ineq	-0.7176481	200966.9
Prob	8.6014880	799490.2
Time	0.3303404	168370.2

rf_crime2



Using ten predictors, the model has higher %IncMSE and IncNodePurity than those of the model with five predictors. However, it sacrifices a lot of percent in R-squared. Therefore, I think the model with five predictors is better.

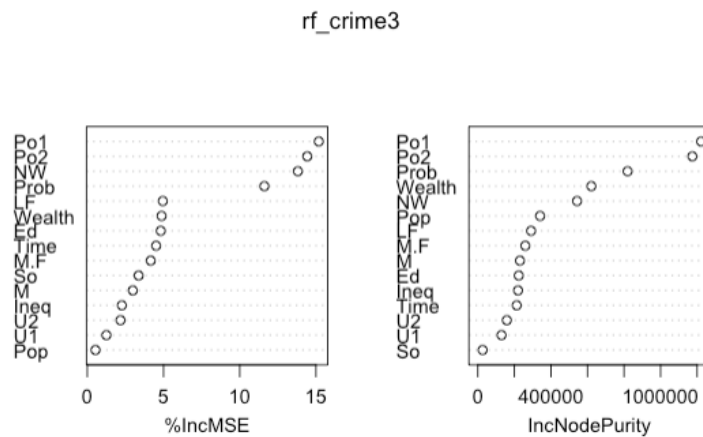
IX. Finally, I will use five predictors but change the number of trees in the forest. Here is the output:

```
Call:
randomForest(formula = Crime ~ ., data = crime, mtry = num_pred, importance = TRUE, ntree = 1000)
Type of random forest: regression
Number of trees: 1000
No. of variables tried at each split: 5

Mean of squared residuals: 83083.47
% Var explained: 43.25
```

X. I used importance() and varImpPlot() on this model.

	%IncMSE	IncNodePurity
M	2.9872559	230565.0
So	3.3662370	26251.4
Ed	4.8181255	223916.8
Po1	15.1855631	1222064.2
Po2	14.4287698	1173993.6
LF	4.9530725	291483.0
M.F	4.1600348	259714.1
Pop	0.5356809	340725.8
NW	13.8151451	543030.1
U1	1.2505434	129714.5
U2	2.1770918	158679.4
Wealth	4.8727746	621342.5
Ineq	2.2663492	219949.6
Prob	11.6145842	819475.2
Time	4.5130966	212778.6



I've tested multiple number of trees in the forest, and it seems like I get the best performance from when `ntree = 1000`. The R-squared is valued at 43.25%, which is the highest I have gotten so far. Additionally, the overall %IncMSE is higher than the other models. The IncNodePurity is lower than the previous two models, but it is still high enough to reduce the prediction error within the trees. Therefore, the best random Forest model is when we use five predictors and 1000 trees, and the important variables are Po1, Po2, NW, and Prob.

Question 10.2

Describe a situation or problem from your job, everyday life, current events, etc., for which a logistic regression model would be appropriate. List some (up to 5) predictors that you might use.

From an academic perspective, we could use a logistic regression model where the response variable is whether a student passes an exam (1) or a student does not pass the exam (0). Some predictors that we may use are amount of hours spent studying, percentage of class attendance, number of office hours attended, average homework grade percentage, and number of hours slept.

Question 10.3

- Using the GermanCredit data set `germancredit.txt` from <https://archive.ics.uci.edu/static/public/144/statlog+german+credit+data.zip/> (description at <https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data>), use logistic regression to find a good predictive model for whether credit applicants are good credit risks or not. Show your model (factors used and their coefficients), the software output, and the quality of fit. You can use the `glm` function in R. To get a logistic regression (logit) model on data where the response is either zero or one, use `family=binomial(link="logit")` in your `glm` function call.

- I. First, I looked at the dataset, specifically column V21 because that is the response variable we are looking at. The values are either 1 or 2. We need to change it to 0 or 1 to be able to use logistic regression on it. Therefore, I changed the value 1 to 0 and 2 to 1.

V21
<dbl>
0
1
0
0
1
0

- II. Next, I created a training and testing dataset. The training dataset has 80% of the data, and the testing dataset has 20% of the data.
- III. I used `glm()` to create the model, set family to `binomial(link = 'logit')`, used the training dataset, and got the summary of the model. Here is the output:

```
Call:
glm(formula = V21 ~ ., family = binomial(link = "logit"), data = german_train)

Coefficients:
(Intercept) 3.978e-01 1.249e+00 0.318 0.750139
V1A12 -2.701e-01 2.437e-01 -1.108 0.267784
V1A13 -9.306e-01 4.018e-01 -2.316 0.020567 *
V1A14 -1.737e+00 2.686e-01 -6.465 1.02e-10 ***
V2 2.893e-02 1.042e-02 2.777 0.005479 **
V3A31 2.255e-01 6.289e-01 0.359 0.719936
V3A32 -7.639e-01 4.753e-01 -1.607 0.108822
V3A33 -9.172e-01 5.233e-01 -1.753 0.079627 .
V3A34 -1.487e+00 4.907e-01 -3.031 0.002440 **
V4A41 -1.832e+00 4.425e-01 -4.141 3.46e-05 ***
V4A410 -1.413e+00 8.263e-01 -1.718 0.087326 .
V4A42 -9.368e-01 2.990e-01 -3.134 0.001727 **
V4A43 -9.044e-01 2.799e-01 -3.230 0.001236 **
V4A44 -8.312e-01 8.946e-01 -0.929 0.352807
V4A45 -3.222e-01 6.092e-01 -0.529 0.596843
V4A46 1.688e-02 4.255e-01 0.040 0.968354
V4A48 -2.213e+00 1.219e+00 -1.816 0.069365 .
V4A49 -8.368e-01 3.850e-01 -2.173 0.029760 *
V5 1.138e-04 5.166e-05 2.202 0.027682 *
V6A62 -3.991e-01 3.182e-01 -1.254 0.209771
V6A63 -4.615e-01 4.762e-01 -0.969 0.332404
V6A64 -1.222e+00 5.473e-01 -2.232 0.025592 *
V6A65 -7.093e-01 2.929e-01 -2.421 0.015462 *
V7A72 -2.017e-01 4.948e-01 -0.408 0.683485
V7A73 -3.028e-01 4.706e-01 -0.643 0.519975
V7A74 -1.105e+00 5.113e-01 -2.162 0.030623 *
V7A75 -4.092e-01 4.712e-01 -0.869 0.385102
V8 3.602e-01 9.933e-02 3.626 0.000287 ***
V9A92 -4.434e-01 4.300e-01 -1.031 0.302374
V9A93 -1.230e+00 4.245e-01 -2.897 0.003769 **
V9A94 -4.630e-01 5.119e-01 -0.905 0.365705
V10A102 7.521e-01 4.771e-01 1.576 0.114917
V10A103 -9.329e-01 4.830e-01 -1.931 0.053423 .
V11 3.282e-03 9.850e-02 0.033 0.973420
V12A122 4.101e-01 2.897e-01 1.415 0.156969
V12A123 1.536e-01 2.649e-01 0.580 0.562115
V12A124 7.122e-01 4.714e-01 1.511 0.130827
V13 -1.868e-02 1.055e-02 -1.778 0.076682 .
V14A142 -1.442e-02 4.733e-01 -0.030 0.975695
V14A143 -4.354e-01 2.724e-01 -1.599 0.109919
V15A152 -3.967e-01 2.739e-01 -1.448 0.147576
V15A153 -5.576e-01 5.303e-01 -1.051 0.293871
V16 3.297e-01 2.124e-01 1.552 0.120602
V17A172 5.151e-01 7.807e-01 0.660 0.509351
V17A173 5.655e-01 7.507e-01 0.753 0.451267
V17A174 8.202e-01 7.597e-01 1.080 0.280307
V18 5.065e-01 2.854e-01 1.775 0.075972 .
V19A192 -3.739e-01 2.323e-01 -1.610 0.107489
V20A202 -1.498e+00 8.079e-01 -1.854 0.063779 .
---
Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 975.68 on 799 degrees of freedom
Residual deviance: 705.07 on 751 degrees of freedom
AIC: 803.07

Number of Fisher Scoring iterations: 5
```

Based on this output, the AIC is 03.07, and the residual deviance is 705.07.

- IV. I created a y-hat variable for the logistic regression model by using predict().
- V. I imported the library, pROC, and used the roc() function on the response variable and y-hat variable. This displays the area under the curve.

```
Call:
roc.default(response = german_test$V21, predictor = round(yhat_log))

Data: round(yhat_log) in 139 controls (german_test$V21 0) < 61 cases (german_test$V21
1).
Area under the curve: 0.676
```

- VI. I performed stepwise selection to help me create a better model. This is the summary of the final model it created:

```
glm(formula = V21 ~ V1 + V2 + V3 + V4 + V5 + V6 + V7 + V8 + V9 +
    V10 + V13 + V16 + V18 + V20, family = binomial(link = "logit"),
    data = german_train)

Coefficients:
            Estimate Std. Error z value Pr(>|z|)
(Intercept)  3.937e-01  9.924e-01   0.397 0.691568
V1A12       -2.987e-01  2.378e-01  -1.256 0.209060
V1A13       -9.614e-01  3.951e-01  -2.434 0.014951 *
V1A14       -1.768e+00  2.639e-01  -6.700 2.08e-11 ***
V2           2.997e-02  1.002e-02   2.992 0.002767 **
V3A31        3.533e-01  6.028e-01   0.586 0.557839
V3A32       -8.373e-01  4.679e-01  -1.790 0.073529 .
V3A33       -9.210e-01  5.150e-01  -1.788 0.073710 .
V3A34       -1.575e+00  4.831e-01  -3.260 0.001116 **
V4A41       -1.676e+00  4.290e-01  -3.907 9.35e-05 ***
V4A410      -1.368e+00  7.968e-01  -1.718 0.085888 .
V4A42       -8.492e-01  2.913e-01  -2.916 0.003549 **
V4A43       -9.546e-01  2.748e-01  -3.474 0.000513 ***
V4A44       -9.276e-01  8.784e-01  -1.056 0.290983
V4A45       -4.809e-01  5.949e-01  -0.808 0.418897
V4A46        1.690e-01  4.151e-01   0.407 0.683923
V4A48       -2.211e+00  1.192e+00  -1.855 0.063584 .
V4A49       -8.435e-01  3.752e-01  -2.248 0.024558 *
V5           1.157e-04  4.739e-05   2.442 0.014588 *
V6A62       -3.345e-01  3.079e-01  -1.087 0.277254
V6A63       -4.750e-01  4.765e-01  -0.997 0.318873
V6A64       -1.207e+00  5.288e-01  -2.282 0.022501 *
V6A65       -6.888e-01  2.813e-01  -2.449 0.014326 *
V7A72       -1.436e-01  4.376e-01  -0.328 0.742696
V7A73       -3.323e-01  4.068e-01  -0.817 0.414015
V7A74       -1.085e+00  4.543e-01  -2.387 0.016979 *
V7A75       -3.387e-01  4.154e-01  -0.815 0.414863
V8           3.687e-01  9.631e-02   3.828 0.000129 ***
V9A92       -3.871e-01  4.166e-01  -0.929 0.352768
V9A93       -1.177e+00  4.105e-01  -2.868 0.004131 **
V9A94       -4.581e-01  4.985e-01  -0.919 0.358127
V10A102      8.665e-01  4.696e-01   1.845 0.065026 .
V10A103     -9.132e-01  4.661e-01  -1.959 0.050076 .
V13         -1.993e-02  9.679e-03  -2.059 0.039510 *
V16          3.197e-01  2.044e-01   1.564 0.117744
V18          5.152e-01  2.790e-01   1.847 0.064772 .
V20A202     -1.443e+00  8.023e-01  -1.799 0.072009 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

    Null deviance: 975.68  on 799  degrees of freedom
Residual deviance: 717.52  on 763  degrees of freedom
AIC: 791.52

Number of Fisher Scoring iterations: 5
```

I performed stepwise selection to choose which variables are important based on their AIC values. The selection process chose the variables: V1, V2, V3, V4, V5, V6, V7, V8, V9, V10, V13, V16, V18, V20. Based on the summary of the new model, it has a lower AIC compared to the original model, valued at 791.52. However, it has a slightly higher residual deviance. Additionally, the gap

V. I calculated $\hat{y}$ for the new model and performed `roc()` to find the area under the curve.

```
Call:
roc.default(response = german_test$V21, predictor = round(yhat_log1))

Data: round(yhat_log1) in 139 controls (german_test$V21 0) < 61 cases (german_test$V21 1).
Area under the curve: 0.655
```

Stepwise selection focuses on model selection and prioritizes lower AIC. However, the AUC is lower than the AUC of my original logistic regression model. We want to focus on higher AUC because we want to find a good predictive model, so predictive accuracy should be the focus. Therefore, I chose to stick with my original logistic regression model.

2. Because the model gives a result between 0 and 1, it requires setting a threshold probability to separate between “good” and “bad” answers. In this data set, they estimate that incorrectly identifying a bad customer as good, is 5 times worse than incorrectly classifying a good customer as bad. Determine a good threshold probability based on your model.

- I. Since my AUC was 67.6%, I used a testing threshold at 0.6. I created a confusion matrix where y-hat threshold was when y-hat was greater than 0.6.

```
yhat_thresh  0  1
           0 124 35
           1  15 26
```

- II. I created variables TP, TN, FP, FN and assigned the value for each one.

```
True Positive = 26
True Negative = 124
False Positive = 15
False Negative = 35
```

- III. I calculated the specificity, sensitivity, precision, and accuracy.

```
Accuracy = 75 %
Sensitivity = 42.623 %
Specificity = 89.209 %
Precision = 63.415 %
```

The question states that incorrectly identifying a bad customer as good is five times worse than incorrectly classifying a good customer as bad. From these performance metrics, specificity is the most important because it emphasizes how many bad customers we correctly reject. The specificity is 89.209%.

- IV. Now, let's try to find an ideal threshold to minimize the total cost as much as possible.

```

```{r}
thresholds <- seq(0.1, 0.9, 0.01)
costs <- sapply(thresholds, function(t) {
 preds <- ifelse(yhat_log > t, 1, 0)
 cm <- table(preds, german_test$V21)
 FP <- cm[2, 1]
 FN <- cm[1, 2]
 5*FP + 1*FN
})
best_threshold <- thresholds[which.min(costs)]
best_threshold
```

[1] 0.86

```

- V. The ideal threshold is .86. Let's try creating another confusion matrix with the new threshold.

```

yhat_thresh1  0  1
0 137  57
1   2   4

```

- VI. Here are the new true positives, true negatives, false positives, and false negatives.

```

True Positive = 4
True Negative = 137
False Positive = 2
False Negative = 57

```

- VII. Finally, I calculated the new accuracy, sensitivity, specificity, and precision.

```
```{r}
accuracy1 = (TP1 + TN1) / (TP1 + TN1 + FP1 + FN1)
sensitivity1 = TP1 / (TP1 + FN1)
specificity1 = TN1 / (TN1 + FP1)
precision1 = TP1 / (TP1 + FP1)

cat('Accuracy = ', round(accuracy1 * 100, 3), '%\n')
cat('Sensitivity = ', round(sensitivity1 * 100, 3), '%\n')
cat('Specificity = ', round(specificity1 * 100, 3), '%\n')
cat('Precision = ', round(precision1 * 100, 3), '%\n')
```

Accuracy = 70.5 %
Sensitivity = 6.557 %
Specificity = 98.561 %
Precision = 66.667 %
```

As you can see from these results, the specificity is significantly higher, valued at 98.561%. This means that our model correctly rejects bad customers 98.561% of the time, which is very good since it is almost perfect. Therefore, a threshold of 0.86 is a good threshold to use on a logistic regression model with all the variables.

References:

<https://www.geeksforgeeks.org/machine-learning/confusion-matrix-machine-learning/>

<https://www.r-bloggers.com/2015/09/how-to-perform-a-logistic-regression-in-r/>

AI

<https://www.rdocumentation.org/packages/tree/versions/1.0-45/topics/tree>

<https://www.rdocumentation.org/packages/randomForest/versions/4.7-1.2/topics/randomForest>